

Summary: Simple Diffusion

GitHub repository: <https://github.com/jlvi179/Advanced-Deep-Learning>

What the Notebook Does

This notebook implements a Denoising Diffusion Probabilistic Model (DDPM) from scratch for a simple 1D dataset. The target distribution is a mixture of two Gaussians (means at -4 and 4, standard deviation of 1). The notebook trains a Multi-Layer Perceptron (MLP) to predict the added noise at each diffusion step and visualizes how pure Gaussian noise is iteratively transformed into the target bimodal distribution during the reverse sampling process.

Architecture Choices

The main design choices focus on the simplicity of the 1D problem and the principles of DDPMs:

- **Data formulation:** The input data consists of single scalar values (1D pixel). The neural network receives a 2D input: the noisy sample x_t and the normalized time step t/T .
- **Noise predictor network:** A simple MLP is used with hidden sizes of 64 and ReLU activations. Because the dataset is simply 1D, complex architectures like U-Net are unnecessary. The output is a single value representing the predicted noise ϵ_θ .
- **Forward process:** The variance schedule is fixed at a constant $\beta = 0.02$. The cumulative products $\bar{\alpha}_t$ are precomputed to allow direct sampling of x_t at any arbitrary time step t during training.
- **Training objective:** Following the DDPM paper, the model is trained to minimize the Mean Squared Error (MSE) between the true sampled Gaussian noise $\epsilon \sim \mathcal{N}(0, 1)$ and the predicted noise $\epsilon_\theta(x_t, t)$.
- **Reverse sampling:** The reverse process starts from pure Gaussian noise $\mathcal{N}(0, 1)$ at $T = 250$ and iteratively denoises the samples down to $t = 0$, scaling the predicted noise using the fixed β and precomputed $\bar{\alpha}_t$.

Fine points to watch:

- **Time conditioning:** The time step t is normalized (divided by the maximum steps $T = 250$) before being fed into the MLP. This ensures that the time feature is on a similar scale to the data values, which helps the network learn the time-dependent noise removal process stably.
- **Noise injection during reverse sampling:** In the reverse loop, additional noise $\sqrt{\beta}z$ is injected at each step except the very last one ($t = 1$), which is crucial for the Langevin dynamics underpinning the diffusion process.